

RzIL Uplifting Migration

proposed on: 22.03.2023

Siddharth Mishra

Telegram : <redacted>

Github : <https://github.com/brightprogrammer>

Email : admin@brightprogrammer.in

Blog : <https://brightprogrammer.in>

CV : <redacted>

Abstract

Rizin has had an intermediate language for over a decade. Major architectures are supporting uplifting to ESIL. During the RSoC 2021, the initial version of the new intermediate language, which is based on the BAP's Core Theory was implemented. In the following years it was improved and some of the architectures were ported to use RzIL instead of ESIL. The main goal of this project is to finish the migration of one or more existing architectures that still use ESIL or add a RzIL support for the architectures that hadn't any uplifting at all.

I'd like to uplift MIPS ISA to RzIL for this project.

Motivation

Background :

- I've been working on my personal reverse engineering and binary exploitation tool for the past few months for helping me solve a CTF challenge or analyzing a Malware programmatically/dynamically.

Why Rizin?

- I've read that good reverse engineers are good developers also, and Rizin has a huge and well maintained code base so I believe working on this project will not only improve my development skills but also improve my reverse engineering skills by improving my knowledge of architectures other than x86 and AMD64.
- I'm interested in program analysis and decompilation theory which indirectly implies a need for deep knowledge of intermediate languages.

- I'm planning my way to do some research on code emulation for fuzzing and coverage testing and finding bugs by other program analytic methods by uplifting low level code to an intermediate language.
- Implementing a new IL is hard and time taking so I want to test my knowledge and determination first by uplifting code to RzIL (an already working and tested IL) and gain deep knowledge of how exactly a task like this is accomplished. Once I learn this, I can create my own IL or just use RzIL directly if I find it more useful.
- I believe in contributing to the tools I use. This helps in getting deeper insights to how things are done in the project/tool itself which is sometimes useful when something goes wrong. I can fix it if I know that there is a problem.
- I've learned that to complete this project I'll need to understand Qemu and modify it which makes this project much more interesting, challenging and thrilling to work.

Plans to contribute after GSoC?

I plan to work on Rizin even after GSoC is complete because of these reasons:

1. I've learned so many things (writing tests, tree-sitter, how RzShell works etc...) in less than 10 days by working on Rizin and there is still a huge codebase to explore. I wonder how much I can learn if I work for a longer time.
2. I'll be contributing some code during GSoC, so my idea is that I must be the maintainer of that code. For this I'll have to stay in touch with the community as long as my code is there.

Alternative Project Ideas?

If I fail to get this project then I'd like to work on : [Exploitation Capabilities Improvements](#), which also sounds like a good project because I like binary exploitation.

Specifications

- <redacted> India (**TIMEZONE** : GMT+5:30)
- I'll start working on the project almost three weeks before the official timeline (May 29), in order to distribute the workload efficiently.
- I'll be mainly using GitHub to track my issues but to track updates on project (like how I solve problems) I'll be writing blog posts on my mentioned blog (top).
- Most of the discussion is going on Rizin's Mattermost server so I'll be using that as the primary platform of communication with my mentors.
- I can confidently given 21 to 28 hours per week but this time is flexible so I can push to to 40 or more hours per week during the months of June and July.

Major Goals

I'm taking reference from the project [description page](#) to decide this.

- **Phase I** : Finish the RzIL uplifting for the MIPS and RISC-V with basic instruction level tests.
- **Phase II** : Implement all changes in the analysis code, added more complex integration tests with types analysis.
- **Extended Timeline** :
 - Add support for MIPS and RISC-V in rz-tracetest and run rz-tracetest on MIPS and RISC-V assembly
 - Add API commands in Rizin to view the IL tree
 - Add GUI widget for IL visualization in Cutter
 - Update the Rizin book with RzIL documentation.
 - Work on Optional Goals

Optional Goals

I'd like to see how bin-diffing works in Rizin and if time allows then I'd like to work on improving bin-diffing capabilities in Rizin itself. There are commercial signature files out there like FLIRT for IDA that need a paid tool to generate signatures. I plan to start a Rizin's own signature generator that can be used to identify library functions in stripped binaries and improve analysis.

Uplifting nanoMIPS to RzIL can also be taken as an optional goal after discussing properly with mentors. This will provide almost complete support of MIPS architecture in Rizin's RzIL.

Timeline

I'm taking reference from a previous contributor's timeline and his blog posts and referenced PRs [here](#).

May 7 - May 13 (Week 1, Phase I starts)

- Go through official Instruction Set or Architecture Programmer's manual for MIPS
 - [MIPS32](#)
 - [MIPS64](#)
- Understand the instruction set properly and how the registers, memory, immediate and instruction itself encoding works.
- Start writing helper functions for different instruction classes to fetch immediate, register, memory address etc... Or just the class of the instruction (AFAIK RISC-V has



different instruction classes, namely R, I, S, B, U and J and MIPS is closely related to RISC-V)

May 14 - May 20 (Week 2)

- Write instruction decoders for MIPS32 and start uplifting instructions to RzIL at the same time.

May 21 - May 27 (Week 3)

- Write instruction decoders for MIPS32 and uplift instructions to RzIL at the same time.

May 28 - June 3 (Week 4)

- Write instruction decoders for MIPS64 and uplift instructions to RzIL at the same time.

June 4 - June 10 (Week 5)

- Write instruction decoders for MIPS64 and uplift instructions to RzIL at the same time.

June 11 - June 17 (Week 6)

- Go through official Instruction Set or Architecture Programmer's manual for microMIPS
 - [microMIPS32](#)
 - [microMIPS64](#)
- Understand the instruction set properly and how the registers, memory, immediate and instruction itself encoding works.
- Start writing helper functions for different instruction classes to fetch immediate, register, memory address etc... Or just the class of the instruction (AFAIK RISC-V has different instruction classes, namely R, I, S, B, U and J and MIPS is closely related to RISC-V)
- Start writing instruction decoder and uplift to RzIL at the same time.

June 18 - June 24 (Week 7)

- Write microMIPS instruction decoders and uplift to RzIL at the same time.

June 25 - July 1 (Week 8)

- Write microMIPS instruction decoders and uplift to RzIL at the same time.

July 2 - July 8 (Week 9)

- Write instruction level tests for MIPS32 and microMIPS32

July 9 - July 15 (Week 10, Midterm Evaluation On 14th @ 1800 UTC)

- Write instruction level tests for MIPS64 microMIPS64

July 16 - July 22 (Week 11)

- Buffer week to complete left out tasks.

July 23 - July 29 (Week 12)

- Get rz-tracetest ready for running on MIPS32 and microMIPS32 by adding support for MIPS32 and microMIPS64 in QEMU

July 30 - August 5 (Week 13)

- Get rz-tracetest ready for running on MIPS32 and microMIPS32 by adding support for MIPS32 and microMIPS64 in QEMU

August 6 - August 12 (Week 14)

- Buffer week

August 13 - August 19 (Week 15)

- Run rz-tracetest on MIPS assembly and custom programs

August 20 - August 26 (Week 16, Phase II submission week)

- Add API and commands in Rizin to visualize the IL tree

August 27 - September 2 (Week 17)

- Implement the Cutter widget to visualize the IL tree

September 3 - September 9 (Week 18, Extended Timeline Starts)

- Buffer week and update Rizin Book

September 10 - September 16 (Week 19)

- Integrate analysis with the new RzIL and improve analysis.

September 17 - September 23 (Week 20)

- Integrate analysis with the new RzIL and improve analysis.

September 24 - September 30 (Week 21)

- Work on "Optional Goals" if all above tasks are complete

October 1 - October 6 (Week 22)

- Work on "Optional Goals" if all above tasks are complete

October 7 - October 13 (Week 23)

- Work on "Optional Goals" if all above tasks are complete

October 14 - October 20 (Week 24)

- Work on "Optional Goals" if all above tasks are complete



October 21 - October 27 (Week 25)

- Work on “Optional Goals” if all above tasks are complete

October 28 - November 4 (Week 26, Final Week)

- Buffer week and code review and submission time.

Micro-Tasks and contributions

Microtask on removing **rz core cmd** calls and porting help command to RzShell : [PR#3421](#) that addresses and closes [PR#2834](#) and partially addresses [PR#383](#)

GSoC experience

I haven't participated in GSoC before and Rizin is the only organization I'm applying to this year.